

Abstract

This project involved designing and implementing a **functional Inventory and Warehouse Management System (IWMS)**. The goal was to establish a robust backend for tracking inventory levels across multiple warehouses and managing core master data (Products, Suppliers, Warehouses). The solution uses **MySQL** to enforce business logic through Stored Procedures (for atomic operations like stock transfers) and Triggers (for automated low-stock alerts). A **lightweight Python Flask** application provides the user interface, ensuring the system is accessible, scalable, and maintains high data integrity by centralizing all transactions at the database level.

Tools Used

Category	Tool / Language	Purpose
Database	MySQL	Core data storage, schema definition, business logic (Procedures & Triggers).
Backend / API	Python Flask	Lightweight web framework connecting the frontend to the MySQL database.
Frontend	HTML, CSS, Jinja2, Vanilla JavaScript	Simple, modern user interface, templating, and client-side filtering.
Database Connector	mysql.connector.pooling (Python)	Handles secure, persistent connections between Flask and MySQL.

Steps Involved in Building the Project

Phase 1: Database Foundation and Integrity (MySQL)

- Schema Definition:** Created the core relational structure, including **Products**, **Warehouses**, **Suppliers**, and the critical many-to-many junction table, **Stock**.
- Alert Mechanism:** Implemented the **trg_low_stock_check** trigger. This automates the logging of an alert whenever a product's stock quantity drops below its predefined **ReorderPoint** during a transaction.
- Data Initialization:** Inserted sample records to enable immediate testing and verification of all inventory views.

Phase 2: Stored Procedures for Business Logic

All critical operations were encapsulated in Stored Procedures to guarantee data integrity and consistency.

1. **Core Inventory Flow:** Developed **sp_transfer_stock** (atomic movement between locations), **sp_ReceiveStock** (stock-in), and **sp_DispatchStock** (stock-out). These include mandatory checks for insufficient stock and positive quantities.
2. **Master Data Management (CRUD):** Created specific procedures for **Create, Read, Update, and Delete** for Products, Warehouses, and Suppliers. Deletion logic includes safeguards, such as preventing a supplier from being deleted if products are still linked to them.

Phase 3: Flask Backend and API Integration

1. **API Development:** Established a Python Flask application that connects securely to the MySQL database pool.
2. **Routing and Procedure Execution:** Defined all necessary routes (e.g., **/products**, **/receive**, **/stock_detail/<id>**). The application fetches data using **SELECT** queries for the Dashboard and executes all transactional logic by calling the respective Stored Procedures via **POST** requests.
3. **Robust Error Handling:** Integrated Flask's **flash** mechanism with the database. This allows custom error messages (e.g., "Transfer failed: Insufficient stock") signaled by the MySQL procedures to be displayed clearly to the user.

Phase 4: Frontend UI Development

1. **Interface Structure:** Designed the web interface using modular HTML templates rendered by Jinja2, ensuring a clean, modern aesthetic with simple CSS.
2. **Key Views:** Implemented the main **Dashboard** (showing total stock and alerts), dedicated **CRUD forms** for master data management, the **Receive Stock** form, and the crucial **Stock Detail** page, which allows the user to see the exact quantity of a product held in *each* individual warehouse.
3. **Functionality:** Incorporated basic JavaScript for immediate client-side table filtering, enhancing usability without complex libraries.

Conclusion

The project successfully delivered a complete, full-stack IWMS prototype. By utilizing **MySQL Stored Procedures** to enforce data integrity and a **Python Flask** interface for efficient user interaction, the system achieves **high reliability and operational control**. The modular design, covering full CRUD operations and core inventory movements, provides a robust foundation ready for expansion into a production environment.